Data Structures – Assignment 1
Reichman University, Spring 2026

Due date: Friday, 27.3.26, 23:59.
You can use the automatic extension and submit up to 24 hours late (up to 3 times during the entire semester).

Honor code:

- Do not copy the answers from any source, including the use of generative AI (e.g. ChatGPT).
- You may work in small groups and consult friends and online resources, but take no records from these interactions and write your solutions independently from scratch!
- Do not forget – you are here to learn!

Submission:

- Submit your solution as a single PDF file via Moodle. Other formats will not be graded.
- Typed submissions will get a bonus of 3 points.
- If you choose not to type your solution, make sure the scan looks good. Hard-to-read submissions will not be graded.

Question A: Math Essentials (20 Points)

In this course, you will frequently rely on basic algebraic properties and sequences to analyze the runtime of algorithms. Prove or evaluate the following claims.

1. Find the exact value of x that satisfies the following logarithmic equation. Show your steps:

$$\log_2(x) + \log_4(x) + \log_8(x) = 11$$

Solution: We can use the regular base-change identity, $\log_b(a) = \frac{\log_c(a)}{\log_c(b)}$, to express all terms in base 2. Applying this to $\log_4(x)$ and $\log_8(x)$:

$$\log_4(x) = \frac{\log_2(x)}{\log_2(4)} = \frac{\log_2(x)}{2} = \frac{1}{2}\log_2(x)$$

$$\log_8(x) = \frac{\log_2(x)}{\log_2(8)} = \frac{\log_2(x)}{3} = \frac{1}{3}\log_2(x)$$

Substitute these back into the original equation:

$$\log_2(x) + \frac{1}{2}\log_2(x) + \frac{1}{3}\log_2(x) = 11$$

Factor out $\log_2(x)$:

$$\left(1 + \frac{1}{2} + \frac{1}{3}\right)\log_2(x) = 11$$

Find a common denominator (which is 6) to add the fractions:

$$\left(\frac{6}{6} + \frac{3}{6} + \frac{2}{6}\right)\log_2(x) = 11$$

$$\frac{11}{6}\log_2(x) = 11$$

Multiply both sides by $\frac{6}{11}$:

$$\log_2(x) = 6$$

Convert the logarithmic equation to its exponential form to solve for x:

$$x = 2^6 = 64$$

2. Assume n is a power of 2 ($n = 2^k$ for some integer $k \geq 1$). Evaluate the following summation and express your final answer as a simplified function of n. Show your steps:

$$\sum_{i=0}^{\log_2 n - 1} \frac{n}{2^i}$$

Solution: First, we can factor out n from the summation since it is a constant with respect to the index i:

$$n \sum_{i=0}^{\log_2 n - 1} \left(\frac{1}{2}\right)^i$$

The summation is now a standard geometric series with the first term $a = 1$, common ratio $r = \frac{1}{2}$, and the number of terms is $m = \log_2 n$. The sum of a finite geometric series is given by the formula $S = a\frac{1-r^m}{1-r}$. Applying this formula:

$$\sum_{i=0}^{\log_2 n - 1} \left(\frac{1}{2}\right)^i = 1 \cdot \frac{1 - (1/2)^{\log_2 n}}{1 - 1/2}$$

Note that $(1/2)^{\log_2 n} = \frac{1}{2^{\log_2 n}} = \frac{1}{n}$. Substituting this in:

$$\frac{1 - \frac{1}{n}}{1/2} = 2\left(1 - \frac{1}{n}\right)$$

Finally, multiply this result by the n we factored out earlier:

$$n \cdot 2\left(1 - \frac{1}{n}\right) = 2n - 2$$

3. Simplify the expression $2^{\log_4 x}$ into an expression without exponents or logarithms as the outermost function. Show your steps.

   Solution: Let $y = 2^{\log_4 x}$. We know that $4 = 2^2$, so we can rewrite the base of the logarithm using the base-change property:

   $$\log_4 x = \frac{\log_2 x}{\log_2 4} = \frac{\log_2 x}{2}$$

   Substitute this back into the exponent of our expression:

   $$y = 2^{\frac{\log_2 x}{2}}$$

   Using the exponent power rule $a^{b \cdot c} = (a^b)^c$:

   $$y = \left(2^{\log_2 x}\right)^{\frac{1}{2}}$$

   By the fundamental property of logarithms, $2^{\log_2 x} = x$. Substituting this yields:

   $$y = x^{\frac{1}{2}} = \sqrt{x}$$

Question B: Proof by Induction (Part 1) (30 Points)

1. Consider a model of a viral outbreak. On day $d = 1$, exactly one person (Patient Zero) becomes newly infected. Suppose that any person who becomes newly infected on day d will go on to infect at most 2 new people on day $d + 1$ (and then they recover and infect no one else). Prove by induction on d that the maximum number of newly infected people on day d is at most 2^{d-1}.

   Solution: **Base case:** For $d = 1$, there is exactly 1 newly infected person (Patient Zero). The formula gives $2^{1-1} = 2^0 = 1$. Since the maximum number of newly infected people is exactly 1, the claim holds for the base case.

Inductive Hypothesis: Assume that for an arbitrary day $k \geq 1$, the maximum number of newly infected people on day k is at most 2^{k-1}.

Inductive Step: We want to prove the claim holds for day $k+1$. By the rules of our model, the people newly infected on day $k+1$ are infected exclusively by the people who were newly infected on day k. Let N_k be the number of newly infected people on day k. By our inductive hypothesis, we know that $N_k \leq 2^{k-1}$. Since each of these N_k individuals can infect at most 2 new people, the number of newly infected people on day $k+1$ (let's call it N_{k+1}) is bounded by:

$$N_{k+1} \leq 2 \cdot N_k$$

Substituting the bound from our inductive hypothesis gives:

$$N_{k+1} \leq 2 \cdot 2^{k-1} = 2^k$$

We can rewrite 2^k as $2^{(k+1)-1}$. Thus, the maximum number of newly infected people on day $k+1$ is at most $2^{(k+1)-1}$, which matches the formula and completes the inductive step.

2. For each of the following proofs, determine whether it is correct or not. For an incorrect proof - explain in 1-2 lines what went wrong.

   (a) **Claim 1:** Given $n \geq 2$ points in the plane, they all lie on the same line.

   Proof: We prove that the claim holds for all integers $n \geq 2$.
   Base case: For $n = 2$, the claim holds.
   Induction Hypothesis: Assume the claim holds for n.
   Induction step: We will show that the claim holds for $n+1$.
   Let $p_1, \ldots, p_n, p_{n+1}$ be any set of points in the plane. Applying the induction assumption on the set $p_1, \ldots, p_n$, we know that $p_1, \ldots, p_n$ lie on a line ℓ_1.
   Applying the induction assumption again on $p_2, \ldots, p_{n+1}$, we get that they lie on a line ℓ_2. Since ℓ_1 and ℓ_2 share the points $p_2, \ldots, p_n$, we conclude that $\ell_1 = \ell_2$, and so $p_1, \ldots, p_{n+1}$ lie on ℓ_1.

   Solution: Claim 1 is incorrect. The error is in the induction step — when $n = 3$, the lines ℓ_1 and ℓ_2 share a single point, p_2. This does not imply that $\ell_1 = \ell_2$ as claimed in the proof.

(b) **Claim 2:** Given the following recursive bound on a function T: $T(2) = 4$ and $T(n) \leq 2T(n/2) + 10n$ for $n = 2^k, k \in \mathbb{Z}^+$, it follows that $T(n) \leq 10n \log_2 n$ for every $n = 2^k, k \in \mathbb{Z}^+$.

Proof: We prove the claim by induction on k.
Base case: For $k = 1$, we have $T(2) = 4 \leq 10 \cdot 2 \log_2 2 = 20$.
Induction Hypothesis: Assume that for $2^{k-1} = \frac{n}{2}$, the claim holds, that is,

$$T\left(\frac{n}{2}\right) \leq 10 \cdot \frac{n}{2} \log_2 \left(\frac{n}{2}\right).$$

Induction step: We will show that the claim holds for n.

(i) $T(n) \leq 2T\left(\frac{n}{2}\right) + 10n$ (given recursive bound on T)
(ii) $T\left(\frac{n}{2}\right) \leq 10 \cdot \frac{n}{2} \log_2 \left(\frac{n}{2}\right)$ (by the induction hypothesis)
(iii) $T(n) \leq 2 \cdot 10 \cdot \frac{n}{2} \log_2 \left(\frac{n}{2}\right) + 10n$ (putting (i) and (ii) together)
(iv) $T(n) \leq 10n(\log_2(n/2) + 1) = 10n(\log_2(\frac{n}{2}) + 1) = 10n \log_2 n$ (algebra + (iii))

Solution: Claim 2 is Correct! The induction base, hypothesis, and step are all mathematically sound.

Question C: Proof by Induction (Part 2) (30 Points)

Prove by induction:

1. for all $n \geq 4$: $n! > 2^n$.

Solution: Base Case: For $n = 4$, we have $4! = 24$ and $2^4 = 16$. Since $24 > 16$, the base case holds.

Induction Hypothesis: Let us assume that $k! > 2^k$ for some $k \geq 4$.

Induction Step: We prove that if the induction hypothesis holds for k, it holds also for $k + 1$. From the definition of Factorial (i.e., $k!$):

$$(k + 1)! = (k + 1) \cdot k!$$

Therefore, by the induction hypothesis:

$$(k + 1)! = (k + 1) \cdot k! > (k + 1) \cdot 2^k$$

Since $k \geq 4$, it is certainly true that $k + 1 > 2$. Thus:

$$(k+1) \cdot 2^k > 2 \cdot 2^k = 2^{k+1}$$

We conclude that the claim holds for every $n \geq 4$.

2. for every positive n: $\sum_{i=2}^{n} i \cdot 2^i = 2^{n+1}(n-1)$.

   Solution: Let us first confirm that the equation stands for the trivial case: $n = 1$. At $n = 1$ the sum runs over an empty set (since the starting index $i = 2$ is greater than $n = 1$), therefore:

$$\sum_{i=2}^{1} i \cdot 2^i = 0$$

   On the other hand, the right side evaluates to:

$$2^{1+1}(1-1) = 2^2 \cdot 0 = 0$$

   and the equation stands.

   We will now prove by induction that the equation holds for any positive integer $n \geq 2$.

   Base case: For $n = 2$,

$$\sum_{i=2}^{2} i \cdot 2^i = 2 \cdot 2^2 = 8$$

   Evaluating the right side: $2^{2+1}(2-1) = 2^3 \cdot 1 = 8$, and the equation holds.

   Induction Hypothesis: Let us assume that $\sum_{i=2}^{k} i \cdot 2^i = 2^{k+1}(k-1)$ for some integer $k \geq 2$.

   Induction Step: We prove that if the induction hypothesis holds for k, it holds also for $k + 1$. Let us write down the sum explicitly:

$$\sum_{i=2}^{k+1} i \cdot 2^i = \left(\sum_{i=2}^{k} i \cdot 2^i \right) + (k+1) \cdot 2^{k+1}$$

By the induction hypothesis, this equals:

$$2^{k+1}(k-1) + (k+1) \cdot 2^{k+1}$$

Factoring out 2^{k+1}:

$$2^{k+1}[(k-1) + (k+1)] = 2^{k+1} \cdot 2k = 2^{k+2} \cdot k$$

Notice that $2^{k+2} \cdot k$ can be written exactly as the right side formula for $n = k + 1$:

$$2^{(k+1)+1}((k+1) - 1)$$

Which is what we wanted to show. Therefore, the equation holds for every positive integer n.

3. the correctness of the binary search algorithm, whose pseudocode is given below.

```
1:  function BSearch(x, A, i, j)
2:      if j − i < 0 then
3:          return −1                                  ▷ not in A
4:      m ← ⌊(i + j)/2⌋                                ▷ middle index
5:      if x = A[m] then
6:          return m
7:      else if x < A[m] then
8:          return BSearch(x, A, i, m − 1)
9:      else
10:         return BSearch(x, A, m + 1, j)
```

Guidance: to prove the correctness of the algorithm, prove that given a value x and a sorted array A with n elements, calling BSearch(x, A, i, j) returns an index $i \leq m \leq j$ such that $A[m] = x$, or returns -1 if x is not in the array A. The induction should be on the size of the interval of A on which the function is called. Namely, on $n = j - i + 1$. Your proof should refer to specific lines of the pseudocode.

Solution: We prove the correctness of BSearch by induction on the length $n = j - i + 1$ of the interval of A on which the function is called. **Induction base:** $n = j - i + 1 = 0$ that is, $j = i - 1$, so the function is called on an empty interval and should hence return -1. Indeed, the

condition in line 2 is satisfied in this case, and -1 is returned in line 3.
Induction hypothesis: Assume that for some $k > 0$, the function returns, for all $n = j - i + 1 < k$, an index m satisfying (i) $i \leq m \leq j$, and (ii) $A[m] = x$, or $m = -1$ if A does not contain the value x in the interval $[i, j]$.
Induction step: We prove that the function is correct for $0 < n = j - i + 1 = k$. Since $j - i > -1$, the condition in line 1 is false. So in line 4, m is assigned a value satisfying $i \leq m \leq j$. If $A[m] = x$ then indeed, in line 6 the index m returned satisfies both conditions. Otherwise, if $x < A[m]$ then, since A is sorted, if x appears in $A[i, j]$, it appears at an index strictly smaller than m. Hence, the call in line 8 to BSearch$(x, A, i, m - 1)$ returns a correct index by the inductive hypothesis, as $(m - 1) - i + 1$ is strictly smaller than $j - i + 1 = k$. Similarly, if $x > A[m]$, then, since A is sorted, if x appears in $A[i, j]$, it appears at an index strictly greater than m. Hence, the call in line 10 to BSearch$(x, A, m+1, j)$ returns a correct index by the inductive hypothesis, as $j - (m + 1) + 1$ is strictly smaller than $j - i + 1 = k$. We have shown that a correct answer is returned in all cases, which completes the proof of the induction step.

Question D: Data Structures (20 Points)

In the first lecture, you were introduced to Singly Linked Lists. Design an algorithm that receives a Singly Linked List of n elements. Your algorithm needs to reverse the order of the linked list (i.e., the first element becomes the last, the last becomes the first, and the arrows point in the opposite direction).

1. Describe your algorithm using pseudo-code. You can assume the List elements have standard 'next' and 'value' fields, and you have access to 'L.head'.

   Solution: We can achieve this by iterating through the list and changing the 'next' pointer of each node to point to the node immediately preceding it. We need three pointers to keep track of our position without losing the rest of the list.

   1: **function** ReverseList(L)

```
2:    prev ← null
3:    curr ← L.head
4:    while curr ≠ null do
5:        nextNode ← curr.next          ▷ Save the rest of the list
6:        curr.next ← prev              ▷ Reverse the pointer
7:        prev ← curr                   ▷ Move prev forward
8:        curr ← nextNode               ▷ Move curr forward
9:    L.head ← prev                     ▷ Update head to the new front
```

2. Analyze the worst-case time complexity and space complexity of your algorithm. For full marks, your algorithm must run in $O(n)$ time and use only $O(1)$ auxiliary space (i.e., you cannot copy the elements to an array or another data structure or use recursion).

   Solution: **Time Complexity:** The algorithm uses a single 'while' loop that iterates exactly once for each node in the linked list. Inside the loop, we only perform constant time $O(1)$ assignments. Therefore, for a list of n elements, the worst-case time complexity is $\mathcal{O}(n)$.

   Space/Auxiliary Complexity: The algorithm only allocates three pointer variables ('prev', 'curr', 'nextNode') to manipulate the list in-place. Because it does not create any new nodes or allocate data structures that scale with the input size n, the auxiliary space complexity is strictly $\mathcal{O}(1)$.